Gebze Technical University

Department of Computer Engineering

CSE344 - System Programming

Homework #5 Report

Priority-Based Cargo Delivery Simulation

Student: Halit Batuhan ASLAN

Student ID: 220104004003

Date: May 2026

Contents

1 System Design

The program implements a fixed-size courier pool. All valid orders are read from the input file at startup, inserted into a shared priority queue, and only then courier threads are created. No new courier thread is created while the shift is running.

Each courier repeatedly takes one order from the queue, delivers it by sleeping for the requested simulation time, updates statistics, and then returns to the queue. When the queue is empty and no active delivery remains, all couriers leave the loop and the main thread writes the final statistics file.

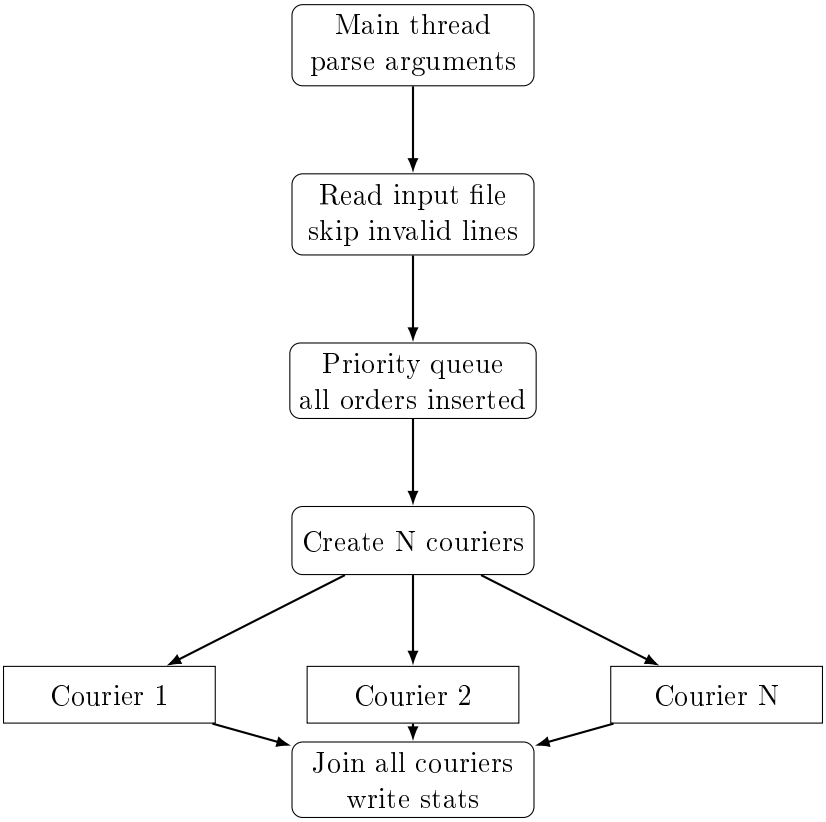

Figure 1: General program flow

1.1 Courier Lifecycle

A courier starts in the depot, locks the queue mutex, and tries to pop the best available order. If it cannot work yet, it waits on the condition variable. After an order is taken, the courier releases the queue mutex before sleeping, so other couriers can continue to take work. At the end of a delivery, the courier updates atomic totals and its own local statistics.

2 Priority Queue Design

The priority queue is implemented as a binary min-heap. The comparison function first compares priority level and then order id. Since EXPRESS is represented with the smallest value, it naturally comes before STANDARD and ECONOMY. If two orders have the same priority, the lower id is placed earlier.

Priority	Internal value	Meaning
EXPRESS	1	Highest priority
STANDARD	2	Normal priority
ECONOMY	3	Lowest priority

Table 1: Priority mapping

The courier logs `DELIVERY_START` immediately after popping the order while still holding the queue mutex. This keeps the visible start order consistent with the scheduling decision.

3 Synchronization Strategy

Primitive	Purpose
`queue_mutex`	Protects the priority queue, active delivery count, and shutdown flag.
`queue_cond`	Blocks idle couriers and wakes them when the shift ends or shutdown starts.
`log_mutex`	Protects every stdout line from interleaving.
`_Atomic` counters	Tracks completed orders, cancelled orders, and total delivery time without a mutex.

Table 2: Synchronization primitives

There is no busy waiting. Couriers use `pthread_cond_wait()` when they need to wait. The main thread uses a timed condition wait only to notice the SIGINT flag without doing unsafe work inside the signal handler.

4 SIGINT Handling

The signal handler only sets a `sig_atomic_t` flag. It does not print, lock a mutex, allocate memory, or cancel threads. The main thread observes this flag in normal control flow and performs the shutdown sequence.

1. SIGINT arrives and the handler sets the flag.
2. The main thread leaves its wait loop.
3. The program enters shutdown mode under `queue_mutex`.
4. Pending orders are removed from the queue and logged as cancelled.
5. Waiting couriers are awakened with `pthread_cond_broadcast()`.
6. Couriers already delivering finish their current order.
7. The main thread joins every courier and writes the final summary.

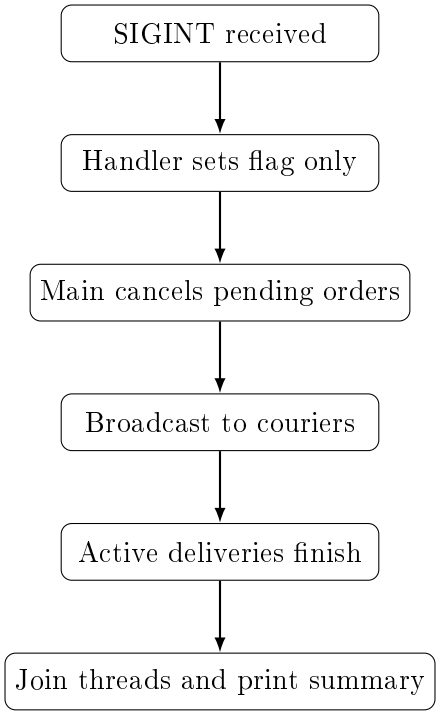

Figure 2: SIGINT shutdown sequence

5 Test Scenarios

The following commands were used for the required scenarios. Each screenshot should show the complete terminal area from the command prompt to program exit.

5.1 Scenario 1: Mixed 10 Orders

```
./cargoGTU -n 3 -i 10orders_mix.txt -s stats.txt
```

This test checks that all EXPRESS orders start before STANDARD and ECONOMY orders. The final summary must show 10 completed and 0 cancelled.

Add the uncut terminal screenshot here.

Expected file: `screenshots/scenario1_mix.png`

Figure 3: Scenario 1 console output

5.2 Scenario 2: Economy Only

```
./cargoGTU -n 10 -i 10orders_economy.txt -s stats.txt
```

This test checks the tie rule. Since all orders have the same priority, DELIVERY_START lines must follow increasing id order.

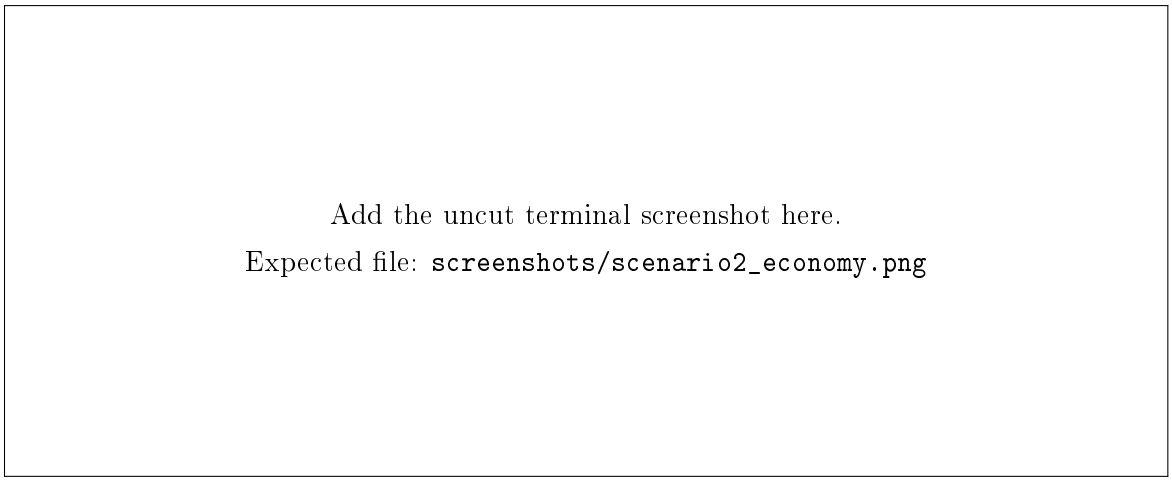

Figure 4: Scenario 2 console output

5.3 Scenario 3: SIGINT

```
./cargoGTU -n 2 -i 20orders_mix.txt -s stats.txt
# send Ctrl+C after about 2 seconds
```

This test checks that active deliveries finish, pending orders are cancelled, and completed plus cancelled equals the total number of valid orders.

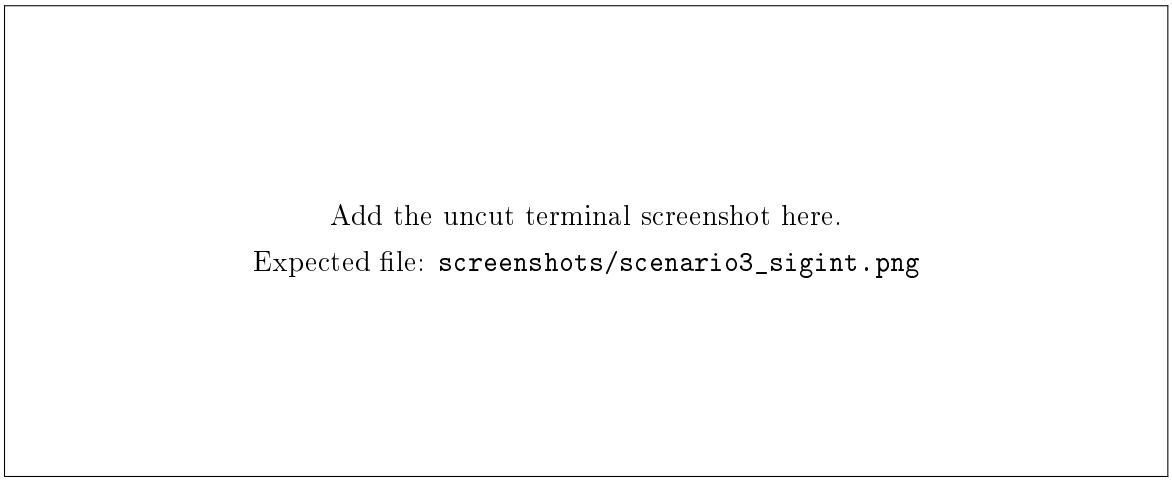

Figure 5: Scenario 3 console output

5.4 Scenario 4: Valgrind

```
valgrind --leak-check=full ./cargoGTU -n 3 -i 10orders_mix.txt -s stats.
    txt
```

This test checks memory cleanup. The expected result is zero errors and zero leaks.

Add the uncut terminal screenshot here.

Expected file: `screenshots/scenario4_valgrind.png`

Figure 6: Scenario 4 Valgrind output

5.5 Scenario 5: High Concurrency

```
./cargoGTU -n 10 -i 20orders_mix.txt -s stats.txt
```

This test is run multiple times. The completed count must stay the same, no order id should appear twice in DELIVERY_START, and priority order must still be preserved.

Add the uncut terminal screenshot here.

Expected file: `screenshots/scenario5_high_concurrency.png`

Figure 7: Scenario 5 console output

6 Challenges and Solutions

6.1 Keeping the visible order correct

With several couriers, it is possible for one courier to pop an order and another courier to print first. To avoid that, the program prints `DELIVERY_START` while the queue mutex is still held. The delivery itself starts after the mutex is released, so concurrency is not blocked for the delivery duration.

6.2 Handling SIGINT safely

Calling complex functions from a signal handler is unsafe in a multi-threaded program. The handler in this implementation only sets a flag. The main thread later cancels pending orders, wakes couriers, joins them, and writes the final files.

6.3 Avoiding duplicate work

Only one courier can pop from the priority queue at a time because the queue is protected by `queue_mutex`. Once an order is removed, it no longer exists in the queue, so another courier cannot take the same order.

7 Conclusion

The implementation uses a fixed thread pool, a protected priority queue, condition variables for blocking, C11 atomic counters for shared statistics, and a clean shutdown path for SIGINT. The code is separated into small modules so parsing, scheduling, synchronization, logging, and statistics writing are easier to follow.